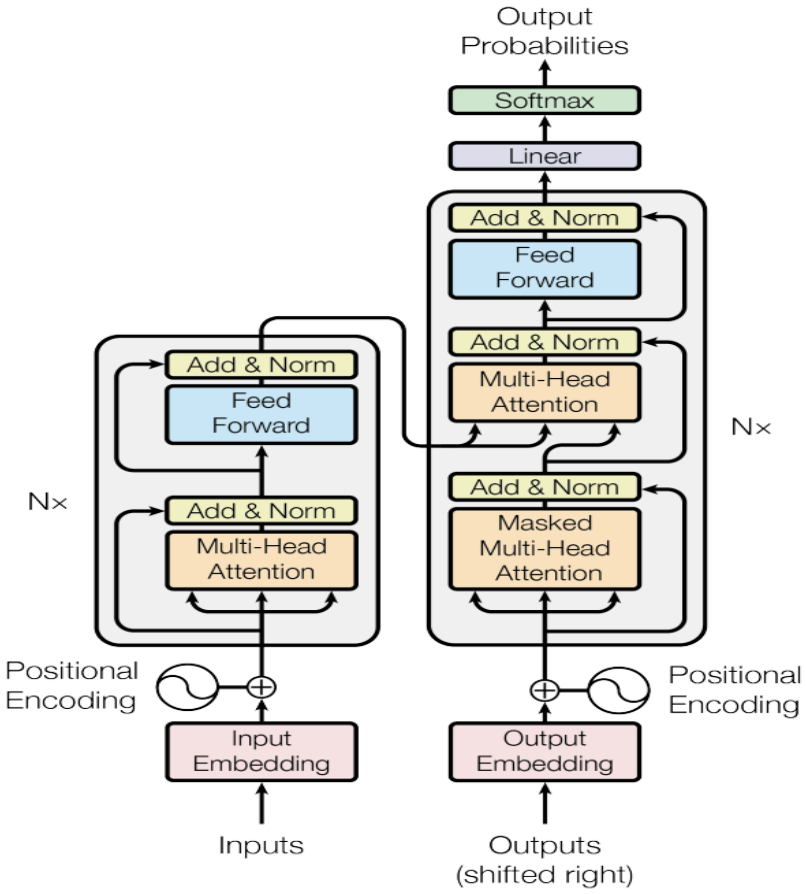
Transformer from scratch



Paper: https://arxiv.org/pdf/1706.03762

Based on original work by: Umar Jamil

GitHub: https://github.com/hkproj/pytorch-transformer/tree/main

Youtube : https://www.youtube.com/watch?v=ISNdQcPhsts&t=20s&ab_channel=UmarJamil

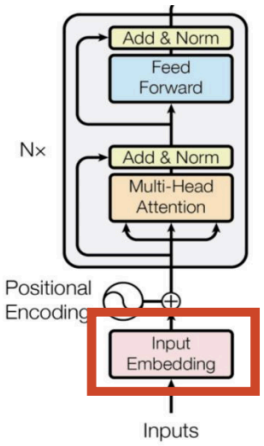
Document compiled by: Santosh Premi Adhikari

Github : https://github.com/santoshpremi/Transformer-from-scratch

Master's Student in Computer Science

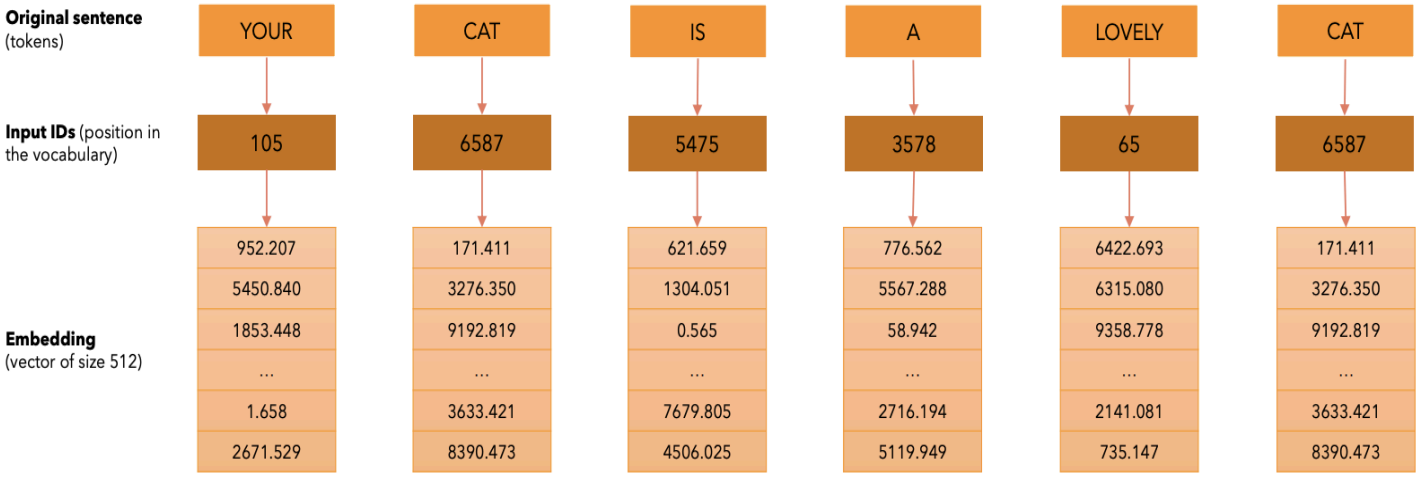
University of Würzburg

1. Input Embeddings



Input Embedding converts words into **vectors** of size **D_model** (usually 512).

1. First, the sentence is turned into **input IDs** (numbers based on vocabulary).
2. Each number is mapped to a **vector of 512 numbers** using an **embedding layer**.
3. **In PyTorch**, use `nn.Embedding(vocab_size, d_model)`.
4. After getting the embedding, multiply it by √**(D_model)** for better scaling (as done in the original Transformer paper).



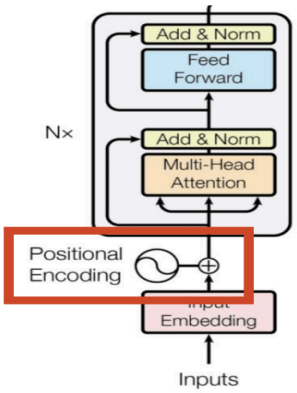

```
class InputEmbeddings(nn.Module):

    def __init__(self, d_model: int, vocab_size: int) -> None:
        super().__init__()
        self.d_model = d_model
        self.vocab_size = vocab_size
        self.embedding = nn.Embedding(vocab_size, d_model)

    def forward(self, x):
        # (batch, seq_len) --> (batch, seq_len, d_model)
        #Multiply by sqrt(d_model)to scale the embeddings according to the paper
        return self.embedding(x) * math.sqrt(self.d_model)
```

2. Positional Encoding

- Embeddings alone don't tell the model **word order**.
- So we **add** special **positional vectors** to embeddings.
- These vectors have size **D_model** (e.g., 512) and
- use **sine** for even positions and **cosine** for odd positions.

Steps:

1. Create a **position vector** for each word (0, 1, 2, ...).
2. Apply **sine** to even indices and **cosine** to odd ones.
3. Add these positional vectors to the word embeddings.
4. Apply **dropout** to prevent overfitting.



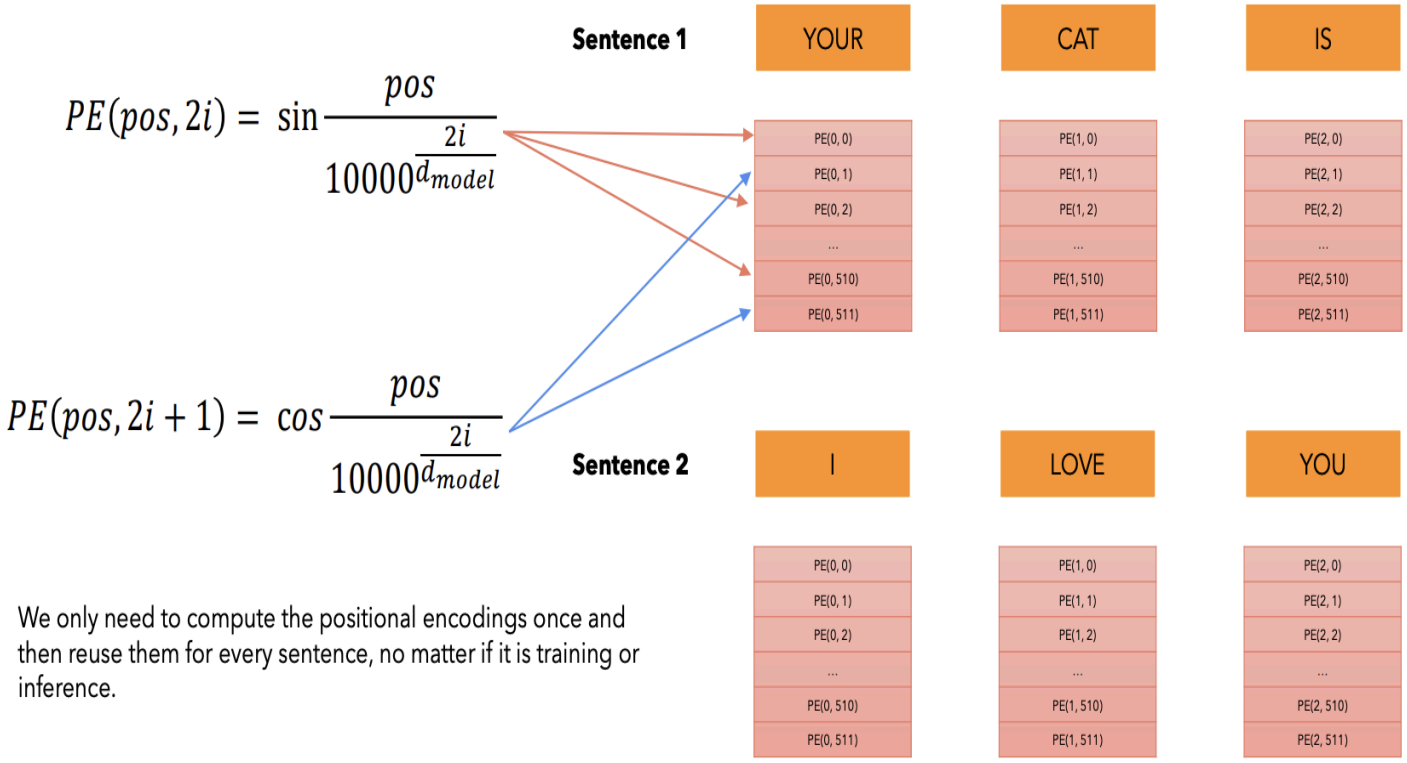
Important: Positional encodings are **fixed** (not learned) but are **saved** with the model.

Original sentence	YOUR	CAT	IS	A	LOVELY	CAT
Embedding (vector of size 512)	952.207 5450.840 1853.448 ... 1.658 2671.529	171.411 3276.350 9192.819 ... 3633.421 8390.473	621.659 1304.051 0.565 ... 7679.805 4506.025	776.562 5567.288 58.942 ... 2716.194 5119.949	6422.693 6315.080 9358.778 ... 2141.081 735.147	171.411 3276.350 9192.819 ... 3633.421 8390.473
+	+	+	+	+	+	+
Position Embedding (vector of size 512). Only computed once and reused for every sentence during training and inference.	... ...	1664.068 8080.133 2620.399 ... 9386.405 3120.159	... ...	... ...	... ...	1281.458 7902.890 912.970 3821.102 1659.217 7018.620
=	=	=	=	=	=	=
Encoder Input (vector of size 512)	... ...	1835.479 11356.483 11813.218 ... 13019.826 11510.632	... ...	... ...	... ...	1452.869 11179.24 10105.789 ... 5292.638 15409.093

```python
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, seq_len: int, dropout: float) -> None:
        super().__init__()
        self.d_model = d_model
        self.seq_len = seq_len
        self.dropout = nn.Dropout(dropout)
        # Create a matrix of shape (seq_len, d_model)
        pe = torch.zeros(seq_len, d_model)
        # Create a vector of shape (seq_len)
        position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1) # (seq_len, 1)
        # Create a vector of shape (d_model)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
         # (d_model / 2)
        # Apply sine to even indices
        pe[:, 0::2] = torch.sin(position * div_term) # sin(position * (10000 ** (2i / d_model))
        # Apply cosine to odd indices
        pe[:, 1::2] = torch.cos(position * div_term) # cos(position * (10000 ** (2i / d_model))
        # Add a batch dimension to the positional encoding
        pe = pe.unsqueeze(0) # (1, seq_len, d_model)
        # Register the positional encoding as a buffer
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + (self.pe[:, :x.shape[1], :]).requires_grad_(False) # (batch, seq_len, d_model)
        return self.dropout(x)
```

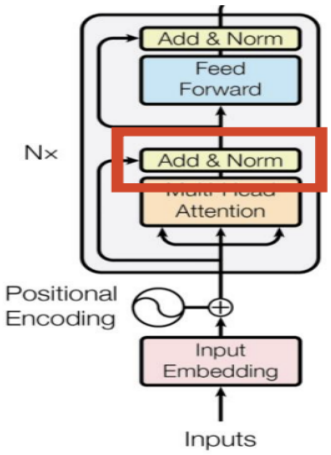
$$PE(pos, 2i) = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

$$PE(pos, 2i+1) = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$



We only need to compute the positional encodings once and then reuse them for every sentence, no matter if it is training or inference.

3. Layer Normalization



- Imagine you have a batch (e.g., 3 sentences, each with numbers/features).
- **Layer Normalization** works **per item** — not across the batch.
- For each item:

1. Calculate the **mean** and **variance** of its features.
2. Normalize:

$$\text{normalized} = \frac{x - \text{mean}}{\sqrt{\text{variance} + \epsilon}}$$

3. **Epsilon (ε)** is a small value (like10^{-6}) added to prevent division by zero and avoid unstable huge numbers.

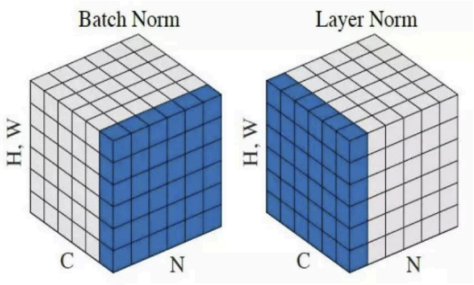
After normalization, two **learnable parameters** are applied:

- **Alpha (γ)**: multiplies the normalized value (scales it).
- **Bias (β)**: adds a value (shifts it).

- Final formula: $\text{output} = \gamma \times \text{normalized} + \beta$

Batch of 3 items

ITEM 1	ITEM 2	ITEM 3
50.147	1242.223	9.370
3314.825	688.123	4606.674
...	...	...
...	...	...
8463.361	434.944	944.705
8.021	149.442	21189.444
μ_1	μ_2	μ_3
σ_1^2	σ_2^2	σ_3^2



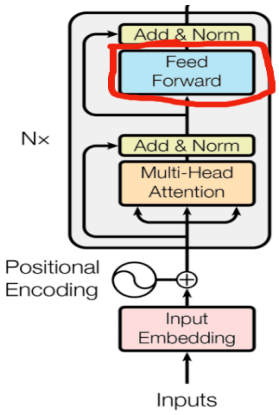
$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

We also introduce two parameters, usually called **gamma** (multiplicative) and **beta** (additive) that introduce some fluctuations in the data, because maybe having all values between 0 and 1 may be too restrictive for the network. The network will learn to tune these two parameters to introduce fluctuations when necessary.

```
class LayerNormalization(nn.Module):

    def __init__(self, features: int, eps:float=10**-6) -> None:
        super().__init__()
        self.eps = eps
        self.alpha = nn.Parameter(torch.ones(features)) # alpha is a learnable parameter
        self.bias = nn.Parameter(torch.zeros(features)) # bias is a learnable parameter

    def forward(self, x):
        # x: (batch, seq_len, hidden_size)
         # Keep the dimension for broadcasting
        mean = x.mean(dim = -1, keepdim = True) # (batch, seq_len, 1)
        # Keep the dimension for broadcasting
        std = x.std(dim = -1, keepdim = True) # (batch, seq_len, 1)
        # eps is to prevent dividing by zero or when std is very small
        return self.alpha * (x - mean) / (std + self.eps) + self.bias
```

4. Feed Forward Block :

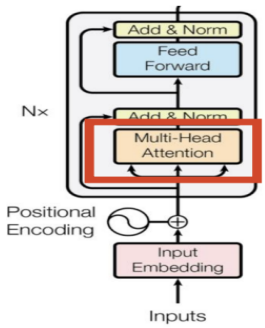
- A **Feed Forward layer** is just **two fully connected layers** with a **ReLU** activation and **Dropout** in between.
- Steps:
  1. Apply a **Linear layer** (W1, B1): from **D_model (512) → D_ff (2048)**.
  2. Apply **ReLU**.
  3. Apply **Dropout**.
  4. Apply a second **Linear layer** (W2, B2):
     from **D_ff (2048) → D_model (512)**.



- In **PyTorch**:
  Use `nn.Linear` for the layers.
  Dropout is optional but helps prevent overfitting.
  Biases (B1 and B2) are included by default in `nn.Linear`.
- **Input shape**: (batch size, sequence length, D_model)
- After passing through the block, the output shape is the same as the input:
  (batch size, sequence length, D_model).

```
class FeedForwardBlock(nn.Module):

    def __init__(self, d_model: int, d_ff: int, dropout: float) -> None:
        super().__init__()
        self.linear_1 = nn.Linear(d_model, d_ff) # w1 and b1
        self.dropout = nn.Dropout(dropout)
        self.linear_2 = nn.Linear(d_ff, d_model) # w2 and b2

    def forward(self, x):
        # (batch, seq_len, d_model) --> (batch, seq_len, d_ff) --> (batch, seq_len, d_model)
        return self.linear_2(self.dropout(torch.relu(self.linear_1(x))))
```

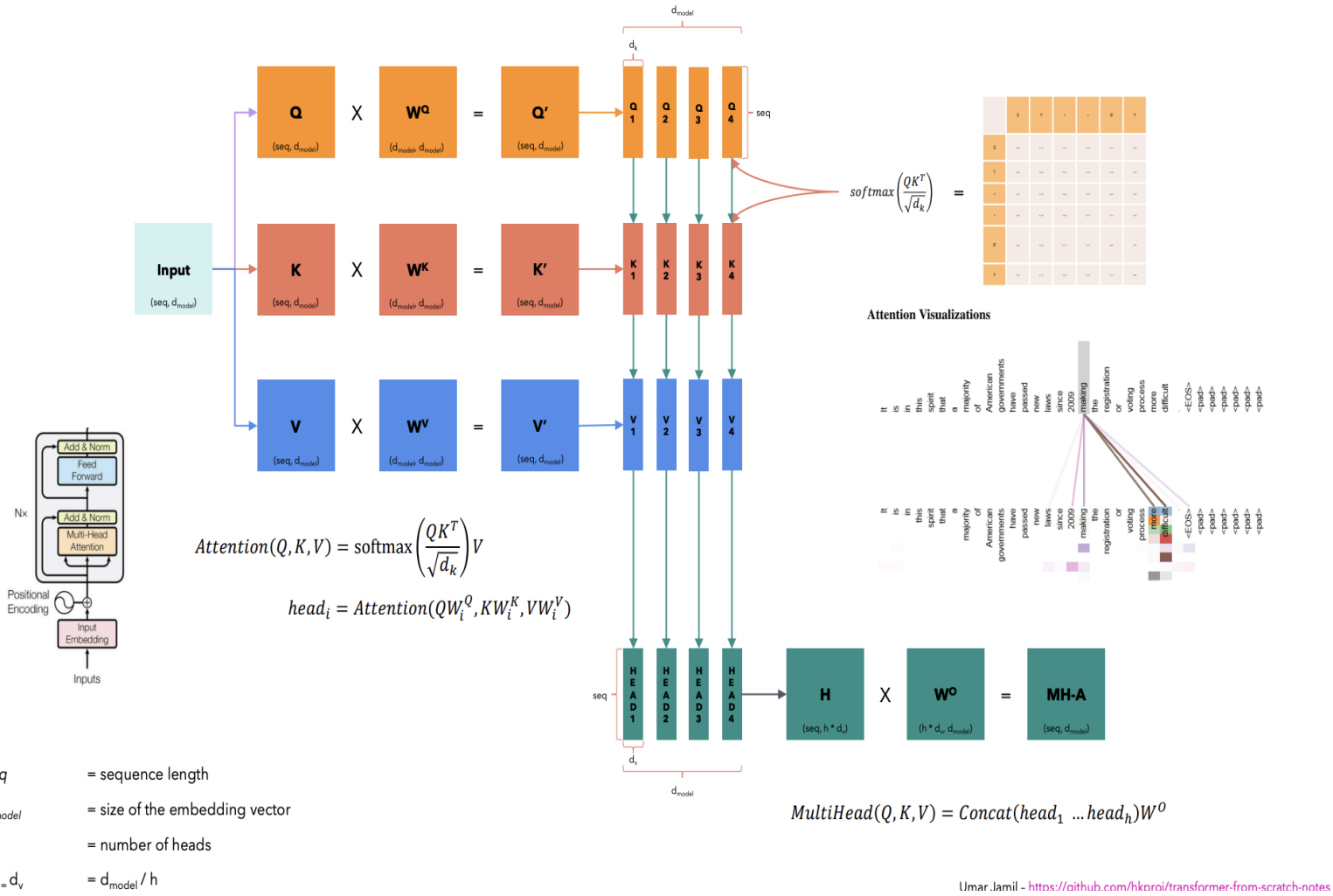
5. Multi-Head Attention



- Multi-Head Attention is **the most important block** in the Transformer architecture.
- It operates by:

  Taking the same input and generating **Queries (Q)**, **Keys (K)**, and **Values (V)**.

  Applying attention mechanisms on different parts of the embeddings simultaneously (multiple heads).



Umar Jamil - https://github.com/hkproj/transformer-from-scratch-notes

Inputs: input tensor of shape (batch_size, sequence_length, d_model).
Steps:

1. Linearly project the inputs into three spaces: **Q**, **K**, and **V** using matrices WQ, WK, and WV.
2. **Split** each of Q, K, V into h parts along the embedding dimension.
   - Each head sees the **full sequence** but **different parts of the embedding**.
3. Apply **scaled dot-product attention** individually for each head.
4. **Concatenate** all heads.
5. Apply a final linear transformation with matrix Wo.

$MultiHead\,(Q,K,V) = Concat\,(head1\,\ldots\,headh\,)Wo$

```python
class MultiHeadAttentionBlock(nn.Module):

    def __init__(self, d_model: int, h: int, dropout: float) -> None:
        super().__init__()
        self.d_model = d_model # Embedding vector size
        self.h = h # Number of heads
        # Make sure d_model is divisible by h
        assert d_model % h == 0, "d_model is not divisible by h"

        self.d_k = d_model // h # Dimension of vector seen by each head
        self.w_q = nn.Linear(d_model, d_model, bias=False) # Wq
        self.w_k = nn.Linear(d_model, d_model, bias=False) # Wk
        self.w_v = nn.Linear(d_model, d_model, bias=False) # Wv
        self.w_o = nn.Linear(d_model, d_model, bias=False) # Wo
        self.dropout = nn.Dropout(dropout)

    @staticmethod
    def attention(query, key, value, mask, dropout: nn.Dropout):
        d_k = query.shape[-1]
        # Just apply the formula from the paper
        # (batch, h, seq_len, d_k) --> (batch, h, seq_len, seq_len)
        attention_scores = (query @ key.transpose(-2, -1)) / math.sqrt(d_k)
        if mask is not None:
            # Write a very low value (indicating -inf) to the positions where mask == 0
            attention_scores.masked_fill_(mask == 0, -1e9)
        attention_scores = attention_scores.softmax(dim=-1) # (batch, h, seq_len, seq_len) # Apply softmax
        if dropout is not None:
            attention_scores = dropout(attention_scores)
        # (batch, h, seq_len, seq_len) --> (batch, h, seq_len, d_k)
        # return attention scores which can be used for visualization
        return (attention_scores @ value), attention_scores

    def forward(self, q, k, v, mask):
        query = self.w_q(q) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
        key = self.w_k(k) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
        value = self.w_v(v) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
        # (batch, seq_len, d_model) --> (batch, seq_len, h, d_k) --> (batch, h, seq_len, d_k)
        query = query.view(query.shape[0], query.shape[1], self.h, self.d_k).transpose(1, 2)
        key = key.view(key.shape[0], key.shape[1], self.h, self.d_k).transpose(1, 2)
        value = value.view(value.shape[0], value.shape[1], self.h, self.d_k).transpose(1, 2)

        # Calculate attention
        x, self.attention_scores = MultiHeadAttentionBlock.attention(query, key, value, mask,self.dropout)

        # Combine all the heads together
        # (batch, h, seq_len, d_k) --> (batch, seq_len, h, d_k) --> (batch, seq_len, d_model)
        x = x.transpose(1, 2).contiguous().view(x.shape[0], -1, self.h * self.d_k)

        # Multiply by Wo
        # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
        return self.w_o(x)
```

What is Self-Attention?

Self-Attention allows the model to relate words to each other.

In this simple case we consider the sequence length **seq** = 6 and $\mathbf{d_{model}} = \mathbf{d_k} = 512$.

The matrices **Q**, **K** and **V** are just the input sentence.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

softmax(**Q** (6, 512) X **K**^{T} (512, 6) / $\sqrt{512}$) =

	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

(6, 6)

* all values are random.

* for simplicity I considered only one head, which makes $d_{model} = d_k$.

How to compute Self-Attention?

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

X **V** (6, 512) = **Attention** (6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

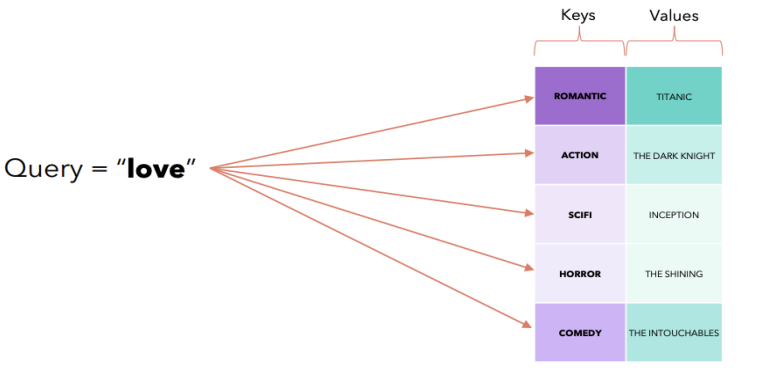
Self-Attention in detail

- Self-Attention is permutation invariant.
- Self-Attention requires no parameters. Up to now the interaction between words has been driven by their embedding and the positional encodings. This will change later.
- We expect values along the diagonal to be the highest.
- If we don't want some positions to interact, we can always set their values to **$-\infty$** before applying the *softmax* in this matrix and the model will not learn those interactions. We will use this in the decoder.

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

Why query, keys and values?

The Internet says that these terms come from the database terminology or the Python-like dictionaries.



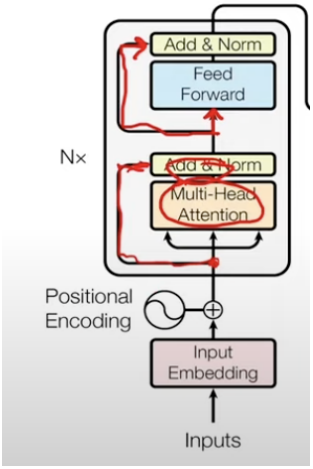
* this could be a Python dictionary or a database table.

6. Residual Connection (Skip Connection)

- Before moving further, there's one last layer we need to build: the residual connection.
- Looking at the architecture:

#We have the output from some layer (e.g., multi-head attention).
#A part of the original input is skipped directly and combined later with the output of the layer.
#This combination happens inside an "Add & Norm" block:

1. The input skips the intermediate layer.
2. The output of the intermediate layer (e.g., multi-head attention) is combined with the skipped input.
3. The combined result is then normalized.



Thus, we need to create a layer that manages this skip connection.
We'll call it a Residual Connection because it's essentially a skip connection.

Implementation Steps:

1. Dropout: To prevent overfitting.
2. Layer Normalization: To stabilize and accelerate training.

We also define the ***forward*** method:
It takes the input x and a sublayer (e.g., multi-head attention or feed-forward layer).

1. Dropout to the output of the sublayer.
2. Adds the original input x.
3. Applies Layer Normalization.

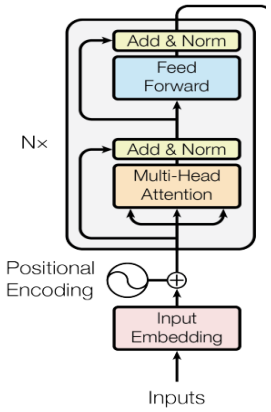
Important Note:

- In the original Transformer paper, they apply the sublayer first, then add and normalize.
- However, many implementations apply normalization first, then the sublayer.
- We'll stick with the version where normalization comes first, as it is common in many practical implementations

```
class ResidualConnection(nn.Module):

        def __init__(self, features: int, dropout: float) -> None:
            super().__init__()
            self.dropout = nn.Dropout(dropout)
            self.norm = LayerNormalization(features)

        def forward(self, x, sublayer):
            return x + self.dropout(sublayer(self.norm(x)))
```

Now we have make all component of Encoder(InputEmbedding, Positional Encoding, layer Normalization, Feed Forward Block, MultiHeadAttentionBlock, Residual Connection) . Let's make an ***Encoder Block.***

7. Encoder Block :



Each encoder block has 2 sublayers:

1. Multi-head self-attention with residual connection
2. Feed-forward network with residual connection

Each sublayer includes:

1. LayerNorm
2. Dropout
3. Residual connection (skip connection)

⚠️ Unlike the original paper (which applies sublayer then norm),
Most implementations apply the norm first, then the sublayer—we follow this.

Self-Attention in Encoder

Uses the same input for query, key, value → hence "self"-attention.
Allows each word to attend to others in the sentence.
Applies a source mask to prevent padding tokens from influencing outputs.

Encoder Block Forward Pass

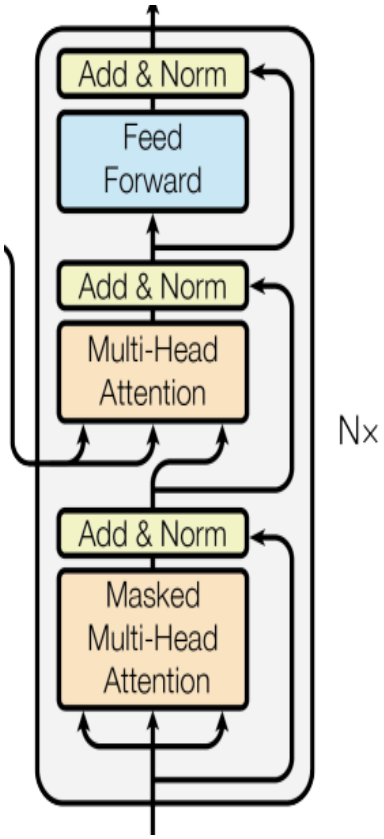
1. Input x → LayerNorm → Self-Attention → Dropout → added to x (residual)
2. Result → LayerNorm → Feed-forward → Dropout → added to input (residual)
3. Both sublayers are passed as lambda functions to residual wrappers.

Encoder (Overall)

Composed of N encoder blocks in sequence.
Output of one is input to the next.
Final output is passed through a LayerNorm before sending to the decoder.

```
class EncoderBlock(nn.Module):

    def __init__(self, features: int, self_attention_block: MultiHeadAttentionBlock, feed_forward_block:
FeedForwardBlock, dropout: float) -> None:
        super().__init__()
        self.self_attention_block = self_attention_block
        self.feed_forward_block = feed_forward_block
        self.residual_connections = nn.ModuleList([ResidualConnection(features, dropout) for _ in
range(2)])

    def forward(self, x, src_mask):
        x = self.residual_connections[0](x, lambda x: self.self_attention_block(x, x, x, src_mask))
        x = self.residual_connections[1](x, self.feed_forward_block)
        return x

class Encoder(nn.Module):

    def __init__(self, features: int, layers: nn.ModuleList) -> None:
        super().__init__()
        self.layers = layers
        self.norm = LayerNormalization(features)
    def forward(self, x, mask):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

8. Decoder Block



Each decoder block has 3 sublayers:

1. Masked self-attention
2. Cross-attention (decoder queries + encoder outputs)
3. Feed-forward network

Each followed by: LayerNorm, Dropout and Residual connection

Self-Attention (Masked)

-Same input used as query, key, and value → "self"-attention.
-Applies target mask to prevent attending to future tokens.

Cross-Attention

-Decoder's output as query, encoder's output as key & value.
-Applies source mask.
-Called "cross-attention" as it links decoder and encoder representations.

Forward Pass of Decoder Block

Inputs:

1. x: decoder input
2. encoder_output: from encoder
3. ***src_mask***, ***tgt_mask***: masks for encoder and decoder inputs

Steps:

1. Self-attention on x with tgt_mask
2. Cross-attention: query = x, key & value = encoder_output, mask = src_mask
3. Feed-forward on result
4. Each sublayer uses residual + norm

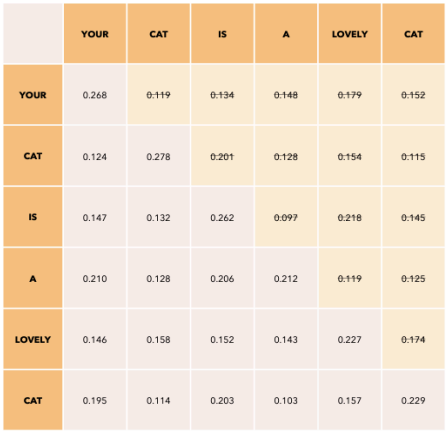
Decoder (Overall)

-Stack of N decoder blocks (ModelList)
-Output of one block → input of the next
-Final output goes through LayerNorm

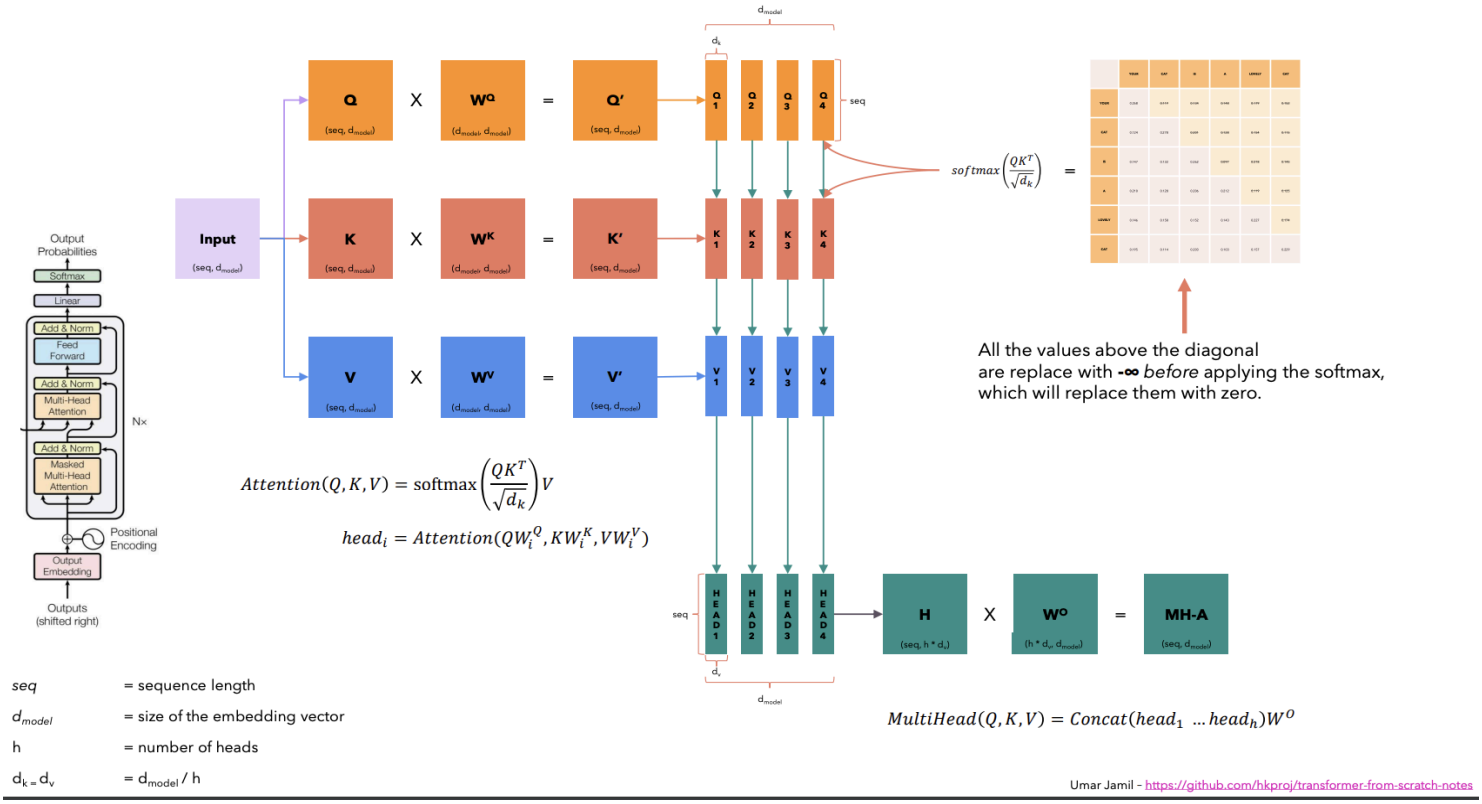
```
class DecoderBlock(nn.Module):

    def __init__(self, features: int, self_attention_block: MultiHeadAttentionBlock,
cross_attention_block: MultiHeadAttentionBlock, feed_forward_block: FeedForwardBlock, dropout: float) ->
None:
        super().__init__()
        self.self_attention_block = self_attention_block
        self.cross_attention_block = cross_attention_block
        self.feed_forward_block = feed_forward_block
        self.residual_connections = nn.ModuleList([ResidualConnection(features, dropout) for _ in range(3)])

    def forward(self, x, encoder_output, src_mask, tgt_mask):
        x = self.residual_connections[0](x, lambda x: self.self_attention_block(x, x, x, tgt_mask))
        x = self.residual_connections[1](x, lambda x: self.cross_attention_block(x, encoder_output,
encoder_output, src_mask))
        x = self.residual_connections[2](x, self.feed_forward_block)
        return x

class Decoder(nn.Module):

    def __init__(self, features: int, layers: nn.ModuleList) -> None:
        super().__init__()
        self.layers = layers
        self.norm = LayerNormalization(features)

    def forward(self, x, encoder_output, src_mask, tgt_mask):
        for layer in self.layers:
            x = layer(x, encoder_output, src_mask, tgt_mask)
        return self.norm(x)
```

What is Masked Multi-Head Attention?

Our goal is to make the model causal: it means the output at a certain position can only depend on the words on the previous positions. The model **must not** be able to see future words.



	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	~~0.119~~	~~0.134~~	~~0.148~~	~~0.179~~	~~0.152~~
CAT	0.124	0.278	~~0.201~~	~~0.128~~	~~0.154~~	~~0.115~~
IS	0.147	0.132	0.262	~~0.097~~	~~0.218~~	~~0.145~~
A	0.210	0.128	0.206	0.212	~~0.119~~	~~0.125~~
LOVELY	0.146	0.158	0.152	0.143	0.227	~~0.174~~
CAT	0.195	0.114	0.203	0.103	0.157	0.229



9. Final Linear (Projection) Layer

After the decoder, we need a final ***Linear layer*** to convert the model outputs (of shape `(batch_size, sequence_length, d_model)`) into logits over the vocabulary (i.e., shape `(batch_size, sequence_length, vocab_size)`).

This layer is necessary because:

- The decoder outputs embeddings of size `d_model`
- We need to map these to vocabulary indices for prediction

So we define a ***Projection Layer***:

- A `Linear(d_model, vocab_size)` layer
- Followed by a `log_softmax(dim=-1)` for numerical stability

***Purpose*:** This layer **projects decoder outputs to the vocabulary space**, making it possible to compute a probability distribution over possible output tokens at each timestep.

```
class ProjectionLayer(nn.Module):

    def __init__(self, d_model, vocab_size) -> None:
        super().__init__()
        self.proj = nn.Linear(d_model, vocab_size)

    def forward(self, x) -> None:
        # (batch, seq_len, d_model) --> (batch, seq_len, vocab_size)
        return self.proj(x)
```

10. Transformer Block Summary

The `Transformer` model is composed of:

- **Components**

1. **Input Embeddings** (`InputEmbeddings`)
   - Converts token indices from source and target vocabularies into `d_model`-dimensional vectors.
2. **Positional Encodings** (`PositionalEncoding`)
   - Adds position information to embeddings, allowing the model to understand token order.
3. **Encoder**
   - Built from `N` repeated `EncoderBlock`s.
   - Each block includes:
     - A multi-head self-attention mechanism.
     - A feed-forward network.
     - Dropout and residual connections.
4. **Decoder**
   - Also made of `N` `DecoderBlock`s.
   - Each block has:
     - A masked multi-head self-attention layer.
     - A multi-head cross-attention layer (attends over encoder output).
     - A feed-forward layer.
5. **Projection Layer** (`ProjectionLayer`)
   - Projects the decoder's output to the target vocabulary space for final predictions.